

1> Explain React.js and its main features. How is React diff from traditional javascript-based web development?

→ React is a javascript library used to create interactive and dynamic user interfaces (UI) for web applications. It was developed by Facebook (meta). React uses reusable components to build the user interface efficiently.

⇒ Main features →

1> Component-Based → React divides the UI into small and reusable components such as Header, Navbar, Button and Footer.

2> Virtual Dom → React uses virtual Dom to update only the required part of the webpage, making the application faster.

3> JSX → Allows writing HTML-like code inside javascript.

4> Reusable Components → Components can be created once and used many times, reducing code.

5> Foster and Efficient → React efficiently manages UI updates and provides a smooth user experience.

React.js	Traditional javascript
→ uses components to build the UI.	usually works directly with HTML elements and Dom.
→ uses Virtual Dom for faster updates.	Directly manipulates the Dom
→ supports JSX for writing UI code.	uses HTML and JavaScript separately.
→ suitable for large and complex application.	suitable for simple websites and small interactions.

//_

⇒ Example →

```
function welcome() {  
  return <h1> Hello, Rishabh! </h1>  
}
```

2) What is JSX in React? Explain the syntax and advantages of JSX with suitable example. Also explain JSX vs normal HTML.

→ JSX (Javascript XML) is a syntax extension used in React.js that allows developers to write HTML-like code inside javascript.

→ JSX makes it easier to create and understand the user interface of React applications.

⇒ Syntax → `const element = <h1> Hello world! </h1>;`

⇒ Advantages of JSX →

1) Easy to understand → JSX uses HTML-like syntax, so it is easy to read.

2) Combines HTML & JS → Allows UI Structure and Javascript logic together.

3) Supports JS Expressions → JS expressions can be written inside.

4) Improves Development → Makes creating React UI simple & ~~easy~~ organized.

5) Works with Components → Helps create and reuse React Components easily.

⇒ Example →

```
function welcome() {
```

```
  const name = "Rishabh";
```

```
  return {
```

```
    <div>
```

```
      <h1> Welcome, {name}! </h1>
```

```
      <p> This is a React component. </p>
```

```
    </div>  
  };
```


JSX

Normal HTML

- | JSX | Normal HTML |
|--|---|
| → Used in React | used to create normal web page |
| → Written inside Javascript | written as a separate HTML document. |
| → uses classname instead of class | uses class |
| → JS can be directly used with {} | Javascript is not directly written this way |
| → usually one Parent element is returned | HTML does not have this React restriction |

3) Explain ~~React~~ the lifecycle of React Component.

- Component lifecycle means the different stages a Component goes through during its life.

1) Mounting → Mounting means the Component is created and displayed on the webpage for the first time.

methods → `render()`, `ComponentDidMount()`

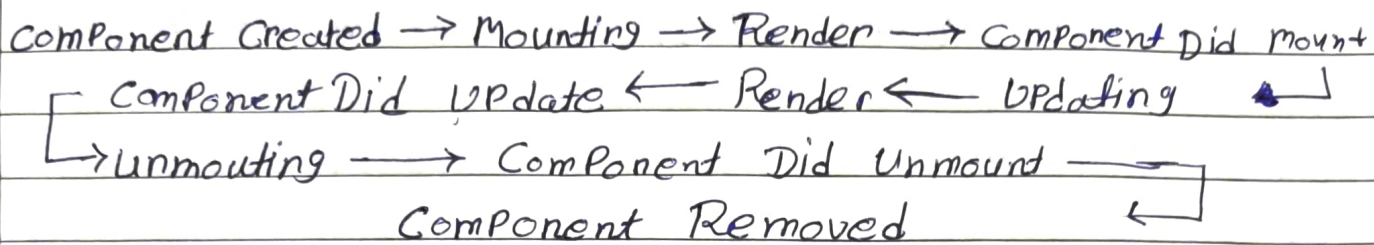
2) Updating → Updating means to Component changes because its state or Props change.

methods → `render()`, `ComponentDidUpdate()`

3) Unmounting → Unmounting means the Component is removed from the webpage.

method → `ComponentDidUnmount()`

⇒ Lifecycle ⇒



4) Explain Redux and Redux Saga, why is Redux used for state management and how does Redux Saga handle asynchronous operations?

→ Redux is library used to manage the state of a React application in one central Place.

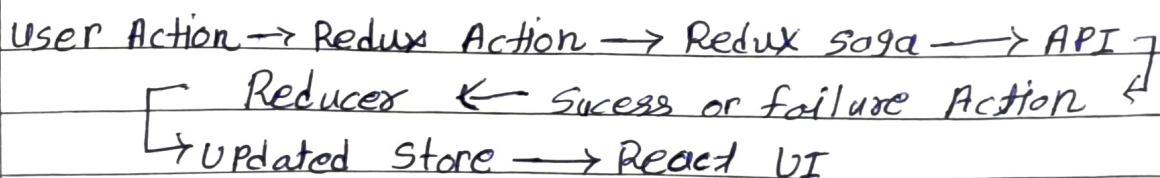
⇒ For example, if many Components need the same data, Redux can store that data in one Place.

⇒ Used :→ It stores application state in one central Place.
→ Different Components can access the same state.
→ It makes State management easier in large applications.
→ changes to state are handled in an organized way.

⇒ Redux Saga

→ Redux saga is middleware used with Redux to handle asynchronous operations such as API calls, data fetching, delays and other side effects.

⇒ work :→



5) How does Redux handle asynchronous function?

→ Redux manages application state. for asynchronous tasks like API calls, Redux can use Redux Saga middleware.

⇒ Work:→

I) user Action:→ user performs an action, like clicking a button.

II) Redux:→ Redux receives the action.

III) Redux saga:→ Saga handles the asynchronous operation.

IV) API call:→ Saga calls the server.

V) Result:→ Saga waits for the API response.

VI) Redux update:→ Saga sends the result to Redux.

VII) state update:→ Redux updates the application state.

⇒ Flow:→

User Action → Redux → Redux saga → API call → Result
State updated ← Redux ←

⇒ Example:→ function* ^{*}getData() {
 const data = yield call(api);
 yield put({type: "success", data});
}

6) Explain React ~~Routing~~ Router. How does routing work in React application?

→ React Router is a library used to create different pages or views in React application.

Ex:⇒ a website can have:

- Home
- About

- Contact

→ React Router allows the users to move between these pages.

//_

=> How does Routing work:

User clicks a link



React Router checks the URL



Finds the matching route



Display the required component

Ex:- import { BrowserRouter, Routes, Route } from
"react-router-dom";

function App() {

 return (

 <BrowserRouter>

 <Routes>

 <Route path="/" element={<h1>Home </h1>} />

 <Route path="/about" element={<h1>About </h1>} />

 </Routes>

 </BrowserRouter>

);

}

7) Explain React Hooks. Discuss the Purpose and usage of useState(), useEffect() and useContext() with example.

→ Hooks are special functions in React that allow functional components to use React features like state and other features.

=> There important Hooks are:

- useState() → manages data

- useEffect() → Performs actions after rendering

- useContext() → gets shared data from context

(I) `useState()` → `useState()` is used to store and change data in a Component.

Example → `const [count, setcount] = useState(0);`

(II) `useEffect()` → `useEffect()` is used to perform an action after the Component renders.

Example → `useEffect(() => {
 console.log("Page loaded");
}, []);`

(III) `useContext()` → `useContext()` is used to get shared data without passing Props through every Component.

Example → `const name = useContext(nameContext);`

Q) What is Node.js?

→ Node.js is a javascript runtime that allows us to run Javascript outside the web browser, mainly on the server.

Ex: → We can use Node.js to create a web server.

1) Javascript Runtime → Node.js allows javascript to run on the server.

2) Event-Driven → Node.js responds to events such as user requests.

3) Non-Blocking → It can handle multiple requests without waiting for each task to finish.

4) NPM → NPM is used to install and manage Node.js Packages.

5) REPL $\rightarrow$ R $\rightarrow$ Read { E $\rightarrow$ Evaluate } P $\rightarrow$ Print } L $\rightarrow$ Loop

It allows us to run js commands directly in terminal

$\Rightarrow$ Simple Architecture $\rightarrow$ Client

$\rightarrow$ Node.js

$\rightarrow$ Event Loop

$\rightarrow$ Response.

9) How to create a simple server using Node.js with Express?

$\rightarrow$ Express.js is a framework for Node.js used to create web servers and web applications easily.

$\Rightarrow$ To create a simple Server:

1) install Express.

2) import Express into to program.

3) Create an Express application.

4) Define a route to handle client requests.

5) start the server using a port numbers.

$\Rightarrow$ Code $\rightarrow$ `const express = require("express");`

`const app = express();`

`app.get("/", (req, res) => {
 res.send("Hello world");
});`

`app.listen(3000);`

Run $\rightarrow$ `npm install express`

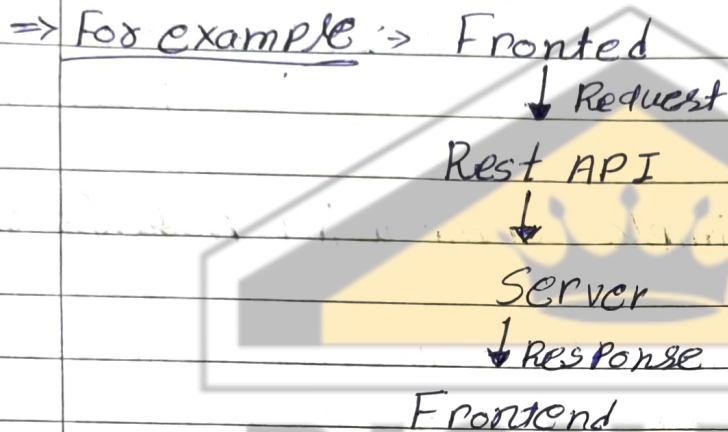
`node server.js`

open $\rightarrow$ `http://localhost:3000`

output $\rightarrow$ Hello world

10) What are Restful APIs? Explain HTTP methods such as GET, POST, PUT, PATCH and DELETE with suitable examples. Also explain the basic of authentication in Rest API.

→ Restful APIs is a way ~~at~~ for applications to communicate with each other over the internet using HTTP.



⇒ HTTP methods ⇒ HTTP methods tell the server what operation we want to perform.

GET ⇒ used to retrieve data

Post ⇒ used to create new data

Put ⇒ used to completely update existing data.

Patch ⇒ used to update only some part of existing data.

Delete ⇒ used to remove data.

⇒ Authentication in REST APIs

→ Authentication means checking whether the user is allowed to access the API

→ For example ⇒ Login → Server check user → Authentication
Token sent with API ← Token given ← successful

→ A common method is using a token or JWT.

11) What are Asynchronous and Synchronous Operations?

⇒ Synchronous ⇒ Synchronous means tasks are performed one after another.

→ The Program waits for the first task to finish before starting the next task.

⇒ Example ⇒ `console.log("start");`
`console.log("Task 1");`
`console.log("End");`

Output ⇒ start
 Task 1
 End

Flow

Task 1

↳ Finish

↳ Task 2

↳ Finish

↳ Task 3

⇒ Asynchronous ⇒ Asynchronous means the Program does not have to wait for a task to finish.

→ It can start another task while the first task is still running.

⇒ Example ⇒ `console.log("start");`
`setTimeout(() ⇒ {`
`console.log("Task 1");`
`}, 2000);`
`console.log("End");`

Output ⇒ start
 End

Task 1

Flow

start task 1

↓

Don't wait

↓

Do task?

↓

Task 1 finish

12> Compare MySQL and MongoDB:

MySQL	MongoDB
→ It is a relational Database	It is non relational Database.
→ Data stored in table	Data stored in documents and Collections.
→ Uses SQL for queries.	Uses JSON-like documents.
→ Has a fixed/structured scheme.	Has a flexible scheme.
→ Relationship are commonly managed using JOINS.	Related data can be embedded in documents.
→ Suitable for highly structured data.	Suitable for flexible and changing data.
→ Example :- Banking, accounting system.	Example :- Real-time apps, Content management.
→ Example :- <code>SELECT * FROM Students;</code>	Example :- <code>{ name: "Rishabh", age: 21 }</code>

13> What is Data migration? Explain the Process of migration data from MySQL to MongoDB. Discuss the challenges involved.

- Data migration means moving data from one database to another database.

→ For example, moving data from MySQL to MongoDB is called data migration.

→ Process of MySQL to MongoDB migration

MySQL → Read data → Convert data → Create MongoDB documents

check data ← Insert data into MongoDB ←

S:→1:→ Read data from MySQL

S:→2:→ Convert MySQL row and columns into MongoDB's document format.

S:→3:→ Arrange the converted data as MongoDB documents.

S:→4:→ store the documents in MongoDB collections.

S:→5:→ check whether the data was transferred correctly

⇒ challenges → Different data structures

→ Relationships

→ Data types

→ Large amount of data

→ data loss/error

14) Explain how database are integrated with Express.js application. Describe the flow from an HTTP request to the database and back to the client.

→ Database integration means connecting an Express.js application with a database so that the application can store and retrieve data.

→ For example, an Express.js application can connect to MySQL or MongoDB.

//_

=> Request-Response Flow

client/Browser

↳ HTTP Request

↳ Express.js Route

↳ Database Query

↳ Database

↳ Result

↳ Express.js

↳ HTTP Response

↳ client/Browser

S:->1> The browser sends an HTTP request to the Express.js server.

S:->2> Express checks the URL and finds the appropriate route.

S:->3> The route sends a query to the database to get, add, update.

S:->4> The database processes the query and sends the result back to Express.

S:->5> Express sends the result back to the client as an HTTP response.

Example:->

```
app.get("/students", async (req, res) => {  
  const data = await db.find();  
  res.json(data);  
});
```

- 15> How do you connect a Node.js application with MongoDB?
- > MongoDB is a NoSQL database used to store data in documents.

→ A node.js application can connect to MongoDB using the MongoDB driver.

- S:->
- 1> Install MongoDB Package
 - 2> Import the MongoDB Package
 - 3> Create a MongoDB client.
 - 4> Connect to MongoDB
 - 5> select the Database
 - 6> Perform database operations.

=> Code:->

```
const { MongoClient } = require("mongodb");  
const client = new MongoClient("mongodb://localhost:27017");
```

```
async function connect() {  
  await client.connect();  
  const db = client.db("school");  
  console.log("MongoDB connected");  
}  
connect();
```

Output:-> MongoDB Connected.